

RNN LSTM and GRU Text Classification Notes

From raw text to token IDs, padding, embeddings, and recurrent classifiers

1. The big picture

A text classifier follows this route: **raw text -> tokenizer -> token IDs -> padding or truncation -> embedding vectors -> RNN, LSTM, or GRU -> sigmoid probability -> label**. The network works with tensors, so words must first become integers and all sequences in a batch must have matching dimensions.

2. Raw data to token IDs

A tokenizer learns a vocabulary by counting words in training text. It assigns each known word an integer ID. For example, the sentence “great movie tonight” may become [41, 98, 305]. These numbers are labels for vocabulary entries, not measurements of meaning. Fit the tokenizer only on training text, then reuse it for validation and test data. An out-of-vocabulary token such as <OOV> handles unseen words.

3. Why records have different lengths

Sentences and reviews naturally have different numbers of words. One message may have 4 tokens and another 120. A neural network batch must be rectangular, so we choose max_len. Short sequences receive padding value 0; long sequences are truncated. Post-padding adds zeros at the end, while pre-padding adds them at the beginning. With an embedding layer, use mask_zero=True so zero padding is ignored.

4. Vocabulary size and embedding

Vocabulary size is the number of token IDs the model can recognize. With vocabulary size 10,000 and embedding dimension 128, the embedding table has shape (10,000, 128): one learned 128-number vector per token. A batch shaped (batch, 200) becomes (batch, 200, 128). This is an embedding lookup, not an outer product.

5. RNN versus LSTM versus GRU

Layer	How it remembers	Use it when
SimpleRNN	One recurrent hidden state	A simple baseline or short dependencies
LSTM	Cell state plus input, forget, output gates	Longer context is important; more capacity is acceptable
GRU	Update and reset gates	You want gated memory with fewer parameters than LSTM

A SimpleRNN can struggle to preserve information over many time steps because gradients may vanish or explode. LSTM and GRU add gates that control what is kept, forgotten, or exposed. In practice, GRU is often a strong efficient default; LSTM remains useful when its extra memory mechanism helps.

6. Output and training

For binary sentiment or spam classification, the final layer is `Dense(1, activation="sigmoid")`. Its output is a probability between 0 and 1. Use binary cross-entropy loss and monitor validation accuracy and validation loss. High training accuracy with weaker validation performance suggests overfitting.

7. Practical checklist

- Fit vocabulary on training data only.
- Use identical tokenizer and padding settings for every split.
- Keep 0 reserved for padding; enable `mask_zero=True`.
- Choose `max_len` by inspecting sequence-length distribution.
- Ensure `input_dim` is larger than every token ID.
- Compare RNN, LSTM, and GRU using validation metrics, not only training accuracy.

Key takeaway

Token IDs identify words. Padding turns unequal-length text records into equal-size batches. Embeddings turn IDs into learned dense features. Recurrent layers use order and context to make the final prediction.